Relatório de Auditoria Técnica e Arquitetural: Aceleração de Algoritmos no Projeto SoC Final

Introdução ao Ecossistema do Projeto System-on-Chip (SoC)

A complexidade crescente das cargas de trabalho computacionais modernas exige uma transição inexorável das arquiteturas de processamento puramente baseadas em software para paradigmas de aceleração em hardware de domínio específico (Domain-Specific Architectures - DSAs). O projeto sob análise, desenvolvido no âmbito laboratorial universitário de *System-on-Chip* (identificado sob a designação 112_SOC_final_project da equipa 07), propõe uma arquitetura de aceleração integrada na plataforma *open-source* Caravel SoC[1]. A premissa central e a audácia arquitetural deste sistema residem na capacidade de descarregar tarefas computacionalmente intensivas da Unidade Central de Processamento (CPU) baseada na arquitetura RISC-V, transferindo-as para blocos de Propriedade Intelectual (IP) dedicados, instanciados fisicamente na área do utilizador, vulgarmente denominada user_project_wrapper[1].

Este paradigma de co-design entre hardware e software procura colmatar a estagnação da Lei de Moore e o estrangulamento da escala de Dennard, recorrendo a silício especializado para maximizar o rácio de operações por Watt e minimizar a latência de execução. A topologia proposta para o projeto baseia-se numa rede de dados rigorosa que liga os IPs de domínio específico a controladores de memória de acesso rápido. O projeto materializa a implementação e otimização de quatro vertentes funcionais distintas mas inter-relacionadas: um filtro de processamento de sinal *Finite Impulse Response* (FIR), um motor de álgebra linear para Multiplicação de Matrizes (*Matrix Multiplication*), um motor de ordenação de dados estruturado no algoritmo *Quick Sort*, e um módulo de transferência assíncrona universal (UART) para comunicação periférica[1].

A finalidade deste documento é fornecer uma auditoria técnica, profunda e exaustiva a cada um dos algoritmos propostos, avaliando não apenas a fundamentação teórica matemática que lhes subjaz, mas primordialmente os desafios colossais de implementação ao nível de Transferência de Registos (RTL) em linguagem Verilog. O hiato entre um algoritmo funcional em software (escrito em C ou Python) e a sua síntese num fluxo de dados em silício obriga a uma reavaliação de conceitos de latência, contrapressão, crescimento de bits e contenção de barramentos. Adicionalmente, este relatório disseca a infraestrutura de comunicação baseada em AXI-Stream, a arbitragem de memória e o firmware de orquestração, oferecendo uma perspetiva holística para garantir a estabilidade, eficiência e segurança forense do circuito integrado final.

Arquitetura de Sistema, Arbitragem e Infraestrutura

de Fluxo de Dados

A viabilidade de múltiplos aceleradores de hardware operarem em paralelo e de forma assíncrona dentro de um System-on-Chip depende inteiramente da robustez da sua hierarquia de memória e do mecanismo de interligação e arbitragem[1]. A arquitetura sob escrutínio desvia a carga do barramento principal através da introdução de um controlador de Acesso Direto à Memória (DMA), que atua como o maestro do fluxo de dados (*data flow*) entre os controladores estáticos (Block RAM) e os referidos motores de aceleração[1].

O Controlador de Acesso Direto à Memória (DMA) e a Norma AXI-Stream

O motor DMA implementado recorre à interface AXI-Stream, uma variante do protocolo Advanced eXtensible Interface da ARM, para gerir transações de dados em contínuo (*streaming*) com largura de banda maximizada. O protocolo AXI-Stream diverge do AXI-Lite ou AXI4 completo por eliminar a necessidade de negociação de endereços físicos em cada ciclo de relógio, baseando-se em vez disso num protocolo de aperto de mão (*handshake*) puramente reativo estabelecido pelos sinais TVALID (emitido pela fonte) e TREADY (emitido pelo destino)[3]. Num pipeline assíncrono perfeitamente otimizado, os dados avançam de um estágio para o outro exclusivamente na borda de subida do relógio em que ambos os sinais se encontram afirmados num estado lógico alto.

No projeto em análise, a configuração do controlador DMA dita as regras operacionais da transferência e a sinalização de estado diretamente para a CPU[1]. A estrutura de configuração de hardware (DMA_cfg) é alocada num endereço base fixo de 0x3000_8000 e encontra-se estruturada de forma concisa em campos de bits específicos para minimizar a pegada de memória e acelerar a descodificação pelo controlador.

Bit(s)	Campo de Registo	Descrição Operacional e Implicações no Fluxo de Dados
[12]	done	Sinaliza ao microprocessador a conclusão bem-sucedida de um bloco de transferência, permitindo o avanço do firmware[1].
[11]	idle	Indica que a máquina de estados finita (FSM) do DMA se encontra inativa, aguardando novas parametrizações[1].

[10]	start	Gatilho lógico acionado exclusivamente pelo firmware para iniciar o fluxo de transações contínuas[1].
[9]	type	Define a polaridade da transação (leitura da RAM para o IP ou escrita do IP para a RAM) e o tipo de carga associada[1].
[8:7]	channel	Multiplexador seletor do canal do IP alvo, permitindo o roteamento exclusivo para FIR, Matmul ou Qsort[1].
[6:0]	length	Define o número de palavras de dados a transferir numa única rajada contínua, limitando o tempo de posse do barramento[1].

O endereço alvo para a iteração física com a memória RAM (addr_DMA2RAM) encontra-se mapeado imediatamente a seguir, no registo 0x3000_8004, utilizando os treze bits menos significativos ([12:0]) para endereçamento físico de blocos de memória[1]. A eficiência transcendental deste bloco provém da eliminação cirúrgica da intervenção da CPU no processo de cópia iterativa de dados. Sem o DMA, a CPU consumiria preciosos ciclos de instrução do RISC-V a executar rotinas elementares de LOAD e STORE, saturando o barramento principal do sistema e neutralizando os ganhos de latência oferecidos pelos aceleradores em silício.

Arbitragem de Memória, Topologia BRAM e Prevenção de Conflitos

A arquitetura do SoC incorpora um módulo Árbitro de Memória (abt), cuja função é essencial para mitigar conflitos de contenção quando o núcleo CPU e o controlador DMA tentam, de forma concorrente, aceder aos Controladores de Memória RAM em Bloco (brc0 e brc1)[1]. O protocolo de transferência estabelece uma matriz de prioridades rígida em que o controlador DMA detém a primazia absoluta nas operações de escrita face aos pedidos iniciados pela CPU[1]. Esta escolha de design reflete o compromisso com a vazão máxima dos aceleradores: estagnar um acelerador em pleno funcionamento devido a uma leitura de rotina da CPU resultaria num colapso da eficiência temporal.

A engenharia por detrás do DMA é notável ao nível da latência, sendo otimizada para que a escrita nos controladores BRAM decorra em zero ciclos de relógio aparentes de atraso ($0T$). Isto significa que as transmissões submetidas pelas portas dma_w_valid e dma_w_data são

absorvidas e registadas pelo controlador de memória na mesma transição ativa da borda de subida do relógio[1]. Adicionalmente, quando o Árbitro se encontra no estado livre (*idle*), os dados de leitura fluem do brc0 para o DMA quase instantaneamente, recorrendo à sinalização dedicada abt_r_ack[1].

No entanto, a auditoria revela que o sistema contempla mecanismos críticos de salvaguarda. Se a CPU introduzir uma interrupção inadiável (como um pedido de extração de dados prioritários para processamento de exceções), a hierarquia força a injeção de ciclos de paragem (*stall cycles*). Este fenómeno materializa-se eletronicamente pela queda abrupta do sinal dma_r_ready, que suspende o avanço do fluxo de dados até que o conflito seja plenamente resolvido pelo pipeline do Árbitro[1]. A capacidade dos aceleradores de hardware para tolerar estas paragens imprevisíveis dita a robustez de toda a arquitetura face à contrapressão dinâmica[3].

Estruturação Espacial e Mapeamento Físico de Memória

A correta alocação física dos recursos determina o isolamento dos domínios operacionais, prevenindo corrupção cruzada de dados por sobreposição de endereços. O sistema em análise estrutura-se num mapa de endereços físicos projetado meticulosamente para os periféricos desenvolvidos na malha do utilizador[1].

Endereço Físico Base	Fim de Bloco de Endereço	Componente de Hardware	Descrição e Propósito Atribuído
0x3800_0000	0x3800_04FF	BRAM_u0	Área designada para os dados iniciais não processados (Raw Data), alimentando os IPs[1].
0x3800_1000	0x3800_1FFF	BRAM_u0	Segmento de memória estática para armazenamento de código de instruções RISC-V[1].
0x3800_7000	0x3800_7FFF	BRAM_u1	Reservatório para deposição dos resultados finais calculados e emitidos pelos

			Aceleradores[1].
0x3000_8000	0x3000_8000	DMA_Controller	Registo de controlo e Configuração primária da máquina de estados do DMA (DMA_cfg)[1].
0x3000_8004	0x3000_8004	DMA_Controller	Registo para o Endereço Base de iteração com a memória RAM (DMA_addr)[1].

Esta separação tangível entre os dados crus de entrada (armazenados entre 0x3800_0000 e 0x3800_04FF) e os resultados consolidados (alojados a partir de 0x3800_7000) em blocos de RAM distintos (BRAM_u0 e BRAM_u1) evoca os princípios de uma arquitetura de Harvard estendida[1]. Ao impedir que as leituras do fluxo de entrada colidam nos mesmos barramentos físicos que as escritas do fluxo de saída, a arquitetura reduz a contenção estrutural, permitindo que os IPs operem com concorrência simétrica nos portos de memória de dupla porta (*dual-port RAM*).

Auditoria Algorítmica I: O Motor de Filtragem FIR (Finite Impulse Response)

A filtragem digital é, inegavelmente, a pedra basilar de qualquer ecossistema de processamento de sinal contemporâneo. O projeto em escrutínio engloba o desenvolvimento e otimização de um acelerador de hardware de domínio específico vocacionado para aplicar um filtro de Resposta ao Impulso Finita (FIR)[1]. Pela sua natureza topológica, um filtro FIR carece integralmente de malhas de retroalimentação (ao contrário dos filtros *Infinite Impulse Response* - IIR). Esta característica confere-lhe uma estabilidade matemática intrínseca em sistemas limitados por aritmética de vírgula fixa (assegurando uma resposta Bounded-Input Bounded-Output, ou BIBO) e permite a síntese de uma resposta de fase perfeitamente linear, vital para evitar a distorção e dispersão de grupo em sinais de alta frequência[4].

Fundamentação Teórica e Equacionamento Analítico

Matematicamente, a operação central de um filtro FIR discreto e invariante no tempo é descrita através da convolução linear clássica entre a sequência de amostras do sinal de entrada, denotada por $x[n]$, e um conjunto estático de coeficientes ou *taps*, representados por $h[k]$. Este conjunto de coeficientes corporiza a resposta ao impulso do filtro e define o seu perfil espetral (passa-baixo, passa-alto, etc.). A equação de diferenças é formulada como:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

Onde N representa o comprimento do filtro (número total de coeficientes) e $y[n]$ consubstancia o valor discreto de saída no instante temporal n[4]. Transportando esta relação analítica para o domínio da frequência discreta através da Transformada Z, obtemos a função de transferência do sistema[4]:

$$H(z) = \sum_{k=0}^{N-1} h[k] z^{-k} = h[0] + h[1]z^{-1} + \cdots + h[N-1]z^{-(N-1)}$$

Nesta formulação, cada elemento multiplicativo associado a z^{-1} não é apenas uma abstração matemática, mas representa fisicamente um atraso estrutural. Na arquitetura de hardware, este atraso consubstancia-se através da instanciação de um registo digital (*flip-flop* ou banco de registos) governado por um sinal de relógio partilhado[7]. Ao contrário das implementações baseadas em software onde o somatório de convolução decorre de forma sequencial num único núcleo acumulador da CPU utilizando aritmética de precisão flutuante da norma IEEE 754, a transposição de um filtro FIR para RTL implica o desenho rigoroso de toda uma via de dados computacional paralela ou semi-paralela.

Crescimento de Bits (Bit-Growth), Quantização e Sobrecarga Aritmética

A migração de modelos teóricos gerados em ferramentas de alto nível (como os algoritmos de síntese fdesign.lowpass ou designfilt do MATLAB) para uma descrição de circuito em Verilog obriga impreterivelmente à conversão dos coeficientes originais de vírgula flutuante infinita para o espartano domínio da vírgula fixa em inteiros[5]. Este processo de truncação e arredondamento introduz invariavelmente "Ruído de Quantização", cujos reflexos na banda de rejeição e na ondulação da banda de passagem (*passband ripple*) devem ser quantificados e minimizados durante o dimensionamento arquitetural.

Para representações limitadas a M bits, o cálculo analítico do coeficiente quantizado b_q (preservando o bit de sinal numa estrutura universal de Complemento para 2) é fidedignamente modelado pela seguinte equação de transformação[4]:

$$b_q = \left\lfloor c \cdot \frac{2^{M-1} - 1}{\max(|c|)} \right\rceil$$

Nesta equação, c representa o coeficiente original concebido em precisão teórica plena[8]. Durante a fase operacional da convolução em hardware acelerado, o fenómeno de *bit-growth* (crescimento algorítmico da dimensão binária da palavra) afirma-se como uma das ameaças

mais severas, e frequentemente negligenciadas, à integridade computacional e ao fecho temporal do circuito. A multiplicação de um dado digital de entrada contendo uma resolução base de L bits por um coeficiente estático com uma largura de M bits gera, por inerência estrita da aritmética binária, um produto parcial que requer obrigatoriamente $L + M$ bits para evitar qualquer risco de sobrefluxo (*overflow*) ou corte espúrio de dados (*clipping*)[4].

Adicionalmente, devido à necessidade mandatória de acumular a soma de N produtos ao longo de toda a malha transversal do filtro polinomial, a expansão do somador final carece da introdução lógica de bits de guarda (*guard bits*) proporcionais à magnitude logarítmica da cascata de somas. Aplicando os corolários do Teorema da Soma de Ponto Fixo, a largura de bits matematicamente rigorosa requerida para garantir a imunidade total à saturação no estágio final (A_{width}) é definida por[4]:

$$A_{width} \geq L + M + \lceil \log_2 N \rceil$$

Se a implementação do módulo de hardware ignorar este axioma, um pico transitório no sinal de entrada resultará num colapso aritmético (*wrap-around* em complemento para dois), onde grandes valores de amplitude positiva transmutam-se instantaneamente num patamar de extremo negativo, originando disrupções caóticas no sinal filtrado[6].

Auditoria e Estratégias de Integração RTL para AXI-Stream

Sob a perspetiva de arquitetura de integração de silício, a auditoria observa que a coexistência deste filtro multiplicador com a infraestrutura AXI-Stream requer um grau de sofisticação que transcende a mera instanciação de operadores matemáticos. O filtro não dita o ritmo dos dados; está inteiramente subordinado à cadência imposta pelo fluxo de memória[3]. Se a via a jusante do filtro não estiver apta a ingerir a amostra calculada (ou seja, se o sinal TREADY de saída for forçado a um estado baixo), a máquina de estados interna do FIR deve exercer contrapressão em direção a montante, interrompendo imediatamente o avanço do seu próprio pipeline para não perder os produtos parciais em voo[3].

Esta paragem mecânica não é trivial. Requer a distribuição meticulosa de lógicas de inibição de relógio (*clock enable*) ou o recurso a componentes auxiliares conhecidos como *skid buffers* nas fronteiras da interface[3]. Um *skid buffer* atua como uma pequena doca elástica com capacidade de retenção de um ciclo extra, assegurando que o encerramento do circuito combinatório do protocolo não origina longos caminhos críticos que inviabilizem a síntese para altas frequências de relógio[3].

Do ponto de vista estrutural, a escolha entre implementar uma "Arquitetura Direta" (*Direct Form*) e uma "Arquitetura Transposta" (*Transposed Form*) ditará o sucesso térmico e temporal do acelerador[6]. Na forma transposta, a amostra recém-chegada $x[n]$ não persegue passivamente um longo corredor de registos de atraso em série. Em vez disso, é disseminada simetricamente por um barramento de alta dispersão (*fan-out*), multiplicando-se simultaneamente com todos os coeficientes em paralelo. Esta técnica genial permite que o registo acumulador seja ancorado fisicamente logo após o multiplicador, fracionando o *Critical*

Path Delay (Atraso do Caminho Crítico) na cadeia da ferramenta de Síntese RTL (Vivado). Para projetos visando execuções de centenas de Megahertz, esta abordagem revela-se indispensável. Acresce ainda que, se a constelação de coeficientes $h[k]$ for deliberadamente simétrica (para assegurar a desejada fase linear do sinal), a exigência em termos de blocos DSP (*Digital Signal Processing Slices* da FPGA ou ASIC) pode ser amputada para cerca de metade, uma vez que se podem pré-somar amostras do histórico e do presente antes de aplicar uma única operação de multiplicação dispendiosa[9].

Auditoria Algorítmica II: O Acelerador de Multiplicação de Matrizes (Matmul)

O acelerador destinado à Multiplicação de Matrizes (doravante designado e instanciado como *Matmul*) constitui o fulcro da computação de alto desempenho[1]. Desde aplicações primárias em transformações gráficas e visão computacional até algoritmos avançados baseados na inferência de *Deep Neural Networks* (Redes Neuronais Profundas), a multiplicação tensorial domina os requisitos computacionais[14]. A migração cirúrgica desta operação da CPU RISC-V genérica para um domínio de hardware dedicado procura contornar o famigerado esgotamento dos registos vetoriais primários do processador, promovendo um paralelismo temporal e espacial à escala maciça sem as penalizações das interrupções de contexto do sistema operativo.

Modelização Matemática e a Disposição em Arquitetura Espacial

Sob o plano matemático rigoroso, para duas matrizes quadradas densas A e B, ambas de dimensão $n \times n$, o cálculo do produto que origina uma matriz de saída C submete-se ao modelo somatório canónico:

$$C_{i,j} = \sum_{k=1}^{n} A_{i,k} \cdot B_{k,j}$$

Uma implementação sequencial elementar — vulgarmente adotada em implementações precoces de software — possui uma complexidade assimptótica de tempo de execução avaliada em $O(n^3)$ e requere de forma extenuante transferências constantes da memória principal para armazenar cada iteração de somas parciais intermediárias. No enquadramento de integração como Propriedade Intelectual (IP) dedicada através de fluxos AXI-Stream, um acelerador de topo adota preferencialmente o desenho microarquitetural de "Matrizes Sistólicas" (*Systolic Arrays*), notabilizado em plataformas como os *Tensor Processing Units*[14]. Num cenário de arquitetura sistólica em malha bidimensional (*grid topology*), os chamados Elementos de Processamento (PEs) são dispostos segundo os eixos cartesianos no subtrato de silício. Cada PE alberga no seu núcleo uma unidade aritmética densa de Multiplicação e Acumulação (MAC) suportada por registos periféricos imediatos[14]. Os operandos são bombeados coordenadamente a partir das fronteiras norte (matriz B) e oeste (matriz A) em direção a sul e este, respetivamente.

À medida que os ciclos de relógio progridem ritmicamente, cada dado transita localmente de um registo PE para o vizinho adjacente. Esta cascata estruturada assemelha-se a um pulso circulatório (daí a nomenclatura "sistólica"). A colossal vantagem deste mecanismo manifesta-se na drástica supressão de interações com a memória externa: a saída gerada pelo Elemento de Processamento situado nas coordenadas espaciais (i, k) alimenta de forma imediata, furtiva e passiva o seu par adjacente $(i, k+1)$. Consequentemente, o rácio de reutilização local dos dados cresce exponencialmente, preservando a tão requerida energia por bit computado[14].

O Dilema da Garganta Computacional: O Modelo Roofline

A eficiência absoluta e o teto limite de um IP *Matmul* devidamente implementado num System-on-Chip encontram-se indissociavelmente limitados pelos axiomas do "Modelo Roofline"[14]. Este modelo de diagnóstico físico-arquitetural dita inequivocamente se um determinado algoritmo matemático se encontra estrangulado pela incapacidade do canal de memória em fornecer dados atempadamente (classificado como *Memory-Bound*) ou se é barrado pelo limite de comutação aritmética do próprio silício (classificado como *Compute-Bound*)[14].

O eixo definidor deste limiar é a "Intensidade Aritmética" (representada por I), uma métrica que traduz o rácio estatístico entre o número de operações de vírgula flutuante (ou operações elementares MAC em inteiros) e a magnitude absoluta em bytes movimentados a partir do banco principal (no nosso caso, a dualidade Block RAM brc0 e brc1)[1]. O ponto ótimo de transição sistémica a ser almejado no design, denotado como I_{ridge}, materializa-se na intersecção exata entre a máxima largura de banda suportada pelo protocolo e o pico absoluto da capacidade computacional da matriz de PEs em Giga-Operações Por Segundo (GOPS)[14].
A auditoria evidencia que as parametrizações da grelha de canais do barramento DMA, conjuntamente com a latência cirurgicamente imposta de zero ciclos ($0T$) na interface BRAM_u0, devem ser alinhadas rigorosamente para alimentar o núcleo matemático de forma incessante[1]. Para prevenir que a máquina entre num estado anémico por carência de dados, o *firmware* e o controlador lógico devem operar sob uma técnica de divisão espacial conhecida como *Loop Tiling* (ou painéis matemáticos)[14]. O particionamento inteligente segmenta a matriz global em pequenos sub-blocos dimensionados exatamente para a ocupação exaustiva da grelha local dos PEs do hardware.
Contudo, este cenário ideal sofre um colapso imediato se a tipologia dos dados transitar para um modelo de "Matrizes Esparsas" (*Sparse Matrices*), onde a vasta esmagadora maioria dos elementos matriciais reflete o valor nulo[17]. As pesquisas e a literatura confirmam que, perante dados esparsos, as regularidades sistólicas vacilam formidavelmente devido a uma taxa brutal de ineficiência operacional por saltos condicionais; o algoritmo transmuta-se, subitamente, de um problema *Compute-Bound* para um problema agressivamente *Memory-Bound* por dissipar o *bandwidth* a importar e computar a multiplicação infecunda de algarismos zeros[14]. Caso as especificações prevejam cargas de inferência desta natureza, o módulo IP de Matmul tem de

prever estágios heurísticos de compactação de fluxo cruzado para mitigar ciclos em inanição.

Auditoria Algorítmica III: O Processador de Ordenamento Quick Sort

O algoritmo de *Quick Sort*, enquadrado como a terceira valência computacional passível de aceleração via DMA do Caravel SoC, encerra dilemas singulares e impõe restrições severas ao engenheiro de arquiteturas digitais, demarcando cruelmente os mundos antagónicos do software estruturado de alto nível e dos paradigmas do hardware digital subjacente[1].

A Dissonância Computacional do Algoritmo face à Topologia RTL

No domínio acolhedor das arquiteturas de software que executam sob Sistemas Operativos e Processadores Genéricos, a família de ordenação baseada no *Quick Sort* revela-se, por vasta margem, o padrão dourado da indústria para matrizes aleatórias amplas[20]. Este consubstancia a quintessência de um método estratégico de "divisão-e-conquista", gozando de uma complexidade temporal média altamente proficiente e invejável quantificada em $O(n \log n)$, com a capacidade notória de permutar a localização de blocos de memória operando primariamente *in-place*[21].

A medula espinal analítica e matemática da ordenação pressupõe a eleição inicial de um eixo dinâmico (*pivot*) contido no subvetor e a subsequente inspeção exaustiva dos valores circundantes. O modelo força permutações recursivas até que todos os dados marginais à esquerda se tornem estritamente menores que o pivô, replicando algoritmicamente a partição em subgrupos decrescentes em cascata de profundidade[21].

Apesar do seu esplendor e da sua supremacia matemática demonstrada na Máquina de Von Neumann pura (onde a CPU dispõe do auxílio vitalício do OS), a conversão de um *Quick Sort* puro para uma Máquina de Estados Finita (FSM) controladora datapath através da linguagem de descrição de hardware Verilog acarreta uma inadequação arquitetural gravíssima que não pode ser descurada[22]. Os ambientes de processamento de software baseiam-se plenamente na abundância de memória dinâmica associada a um *Stack Pointer* extensível. Isto possibilita que rotinas que se invocam a si próprias possam reter endereços de retorno virtuais nas profundezas da memória quase indefinidamente.

Em contraste cru e intransigente, um circuito digital construído pela compilação hierárquica do código RTL possui uma via de dados que é cimentada de forma fixa e petrificada no instante da Síntese. A linguagem descritiva que modela hardware genuíno rejeita a conceção de chamadas ativas com profundidades não delimitadas estaticamente[22]. Consequentemente, a auditoria postula que para a concretização de um módulo desta envergadura, a máquina será forçosamente reesboçada enquanto "Quick Sort Iterativo". A penalização material de adotar a forma iterativa decorre da obrigatoriedade do próprio criador alocar, instanciar e gerir uma matriz local baseada numa topologia estrita de Pilha de Capacidade Finita (LIFO) para preservar as fronteiras dos sub-vetores na travessia logarítmica[21]. Se a matriz em causa para ordenamento for colossal e o número contido de subdivisões desestabilizar a previsão original, o acelerador entrará num ciclo falível por saturação de *Stack* físico, resultando invariavelmente na perca e na corrupção silente e catastrófica dos dados de base.

O Abismo da Latência, Arbitragem Caótica e Vulnerabilidade a Injeções

Um vetor de fraqueza intrínseco, que diferencia marginalmente o acelerador *Quick Sort* dos equivalentes topologicamente perfeitos para hardware em silício (como redes de *Bitonic Sort* ou metodologias nativamente paralelizadas como o *Odd-Even Merge Sort*[22]), reside na imprevisibilidade aterradora do seu perfil de latência de fecho e nos seus saltos erráticos de endereço[20].

Num cenário laboratorial limpo e pseudorrandómico, as interações progridem eficazmente. Contudo, perante o chamado "Pior Cenário" (Pior Caso Matemático) — um fenómeno que se instala implacavelmente quando os dados maciços injetados pelo pipeline DMA a partir da memória física RAM_u0 já se encontram substancial ou totalmente ordenados, perversamente invertidos, ou na presença de uma escolha sistémica de pivôs subótimos reincidentes —, a matriz fragmenta-se singularmente e o desempenho mergulha abruptamente numa fenda estatística traduzida na ineficiência quadrática máxima de $O(n^2)$ [20].

Esta particularidade não é uma mera nota de rodapé na ciência de computadores; ela dita as normas do congestionamento eletrónico. Durante o processo turbulento e moroso de $O(n^2)$, a quantidade massificada de tráfego injetado no canal de interligação DMA é diametralmente assimétrica[21]. Operando debaixo de infraestruturas normalizadas AXI e do seu barramento Mestre/Escravo de acesso sequencial linear, a máquina efetuará saltos constantes e comparações em índices pontuais com elevadíssima disparidade de distanciamento, obrigando a constantes quebras do pipeline de memória subjacente[1]. O Árbitro do controlador RAM debater-se-á com uma cadência ininterrupta e estocástica de pedidos cruzados de DMA Read e DMA Write fragmentários e inter-relacionados, somando um *overhead* penalizador por cada quebra na rajada sequencial[1].

Além dos obstáculos operacionais em desempenho líquido, do ponto de vista de Segurança Lógica do Silício e da Auditoria Criptográfica Avançada, a implementação direta de módulos não temporizados numa cadência constante acarreta vulnerabilidades sistémicas. Com a variabilidade extrema que separa as complexidades entre $O(n \log n)$ e $O(n^2)$ face à ordenação algorítmica particular do feixe de entrada de dados, o *Quick Sort* introduz implicitamente uma porta entreaberta à execução de ataques de canal lateral baseados no tempo (*Timing Attacks*)[20]. Injetando conjuntos de dados ardilosamente deformados no SoC contendo um perfil patológico, um ator cibernético não autorizado pode estagnar a máquina, prolongando extenuantemente o estado de prioridade do árbitro e saturando as vias, enquanto, colateralmente, infere e deduz perfis estatísticos a partir da assinatura de tempo necessária para que a bandeira de sistema 16'hAB21 da verificação assinale a sua terminação. A contenção desta fragilidade exige que as arquiteturas RTL incorporem metodologias enraizadas de pseudo-aleatoriedade nos contadores de seleção do pivot, possivelmente combinando o algoritmo central com escapes para heurísticas determinísticas de encerramento local por ordenação sequencial estrita, imunizando a malha principal contra picos térmicos agudos[20].

Interface Universal de Monitorização: O Controlador UART

A inclusão de um módulo transceptor assíncrono (UART - *Universal Asynchronous Receiver-Transmitter*) no núcleo vital do System-on-Chip viabiliza de forma tangível a intercomunicação e monitorização em banda lateral (*side-band debugging*) com estações anfitriãs externas. Embora sob um enquadramento pragmático constitua inquestionavelmente o bloco arquitetural de Propriedade Intelectual (IP) com o maior grau de madurez e menor novidade teórica de toda a topologia analisada, a desconsideração deste nó de contacto nas malhas avaliativas do projeto conduz frequentemente a encravamentos crónicos nas comunicações mestre-escravo operadas pelo RISC-V[1].

O mapa analítico e orgânico reservado aos domínios de endereço e registos inerentes ao módulo UART assinala uma premissa de descomplicação estruturada[1]:

- O registo de amostragem ativa e descodificação de receção RX_DATA aloja-se imperativamente no endereço hexadecimal restrito a 0x3100_0000, operando tramas assíncronas serializadas desconstruídas para o vetor estático contido nos bits limítrofes de [7:0].
- Analogamente, a interface impulsionadora das saídas, TX_DATA, encontra o seu porto enraizado em 0x3100_0004, executando tarefas homólogas num barramento confinado de envio sob os mesmos bits de amostragem [7:0][1].
- Contudo, é o terminal compilador de eventos estatísticos fidedignos, materializado conceptualmente num Registo de Controlo de Estado globalmente partilhado — o STAT_REG — posicionado escrupulosamente no índice 0x3100_0008, que exige a vigilância pormenorizada da lógica de sistema[1].

É imperativo decifrar as implicações arquiteturais deste vetor consolidado, sendo que os indicadores armazenados refletem e impulsionam os alertas essenciais canalizados a partir da máquina de estado central do UART com a missão de desencadear trâmites de interrupção submetidos ao microcontrolador ou forçar a criação de ciclos extenuantes baseados em amostragem contínua (mecanismo vulgo *polling*)[1]. Os alarmes críticos abarcados englobam ruturas ou falhas de compasso manifestadas através da deteção de desvios métricos base no enquadramento (as famosas derivas assinaladas na flag do *Frame Error*), o colapso e corrupção das esferas recetoras causados pelo afluxo violento de dados sobrepostos descarregados ininterruptamente (*Overrun Error* originado pela lentidão nas drenagens de software do SoC), juntamente com a assertividade perentória da notificação que acautela a suspensão imediata de carregamento pelo limiar físico absoluto no lado transmissor (*Tx_full*) que paralisa logicamente o ciclo de gravação até o desimpedimento cadenciado na linha[1]. Esta constelação de marcadores exige, indubitavelmente, que a inter-relação sistémica subjacente moldada ao longo do labirinto dos canais do tipo AXI não promova ou valide engarrafamentos cruciais por colapsos e interferências na memória SRAM partilhada decorrente das exigências interativas com a arquitetura periférica; ao bloquear local e contiguamente estes erros, defende-se a priorização na completude imperativa dos aceleradores massivos FIR ou Matmul subjacentes.

Verificação Comportamental, Sequenciamento de Firmware e Matrizes de Rastreio (Checkbits)

O enquadramento canónico desenhado segundo a diretriz basilar da infraestrutura do SoC Caravel estabelece peremptoriamente que qualquer arranjo criativo de blocos e matrizes lógicas instanciados livremente pela comunidade nos portões internos delimitadores da user_project_wrapper não desfruta de soberania executiva espontânea. Estão intimamente sujeitos à criação compulsória de rotinas instrumentais de configuração em *firmware* primário, meticulosamente concebidas em linguagem funcional de baixo nível (como C) e compiladas com a minúcia da cadeia especializada e interplataforma RISC-V (rv32i) para conduzir sem falhas o processador de escalonamento matricial base (neste ecossistema manifestado no núcleo *VexRiscV*)[1].

A fundação orquestradora manifestada no ficheiro basilar de firmware principal adota para si o papel de maestro de eventos assíncronos e desencadeamentos operacionais encadeados debaixo das amarras de um laço de processamento vitalício (*looping* contínuo), cuja magnitude rítmica permanece configurada e indexada no sistema via a imposição rígida da macroestrutura designada por TIMES_RERUN[1].

Neste espetro de comando, os instantes críticos imediatamente subsequentes ao derrubar coordenado da libertação funcional contida no pino de RESET observam o despontar invocativo de rotinas essenciais de ambientação sistémica (sob o patrocínio nominal atribuído pelas invocações mprj_init, seguidas de perto pelos acertos ao analisador embutido por la_init, e a devida formatação interativa referenciada através do mecanismo centralizador em uart_interrupt_init). Somente superado este patamar, o cérebro processador do Caravel SoC principia com a cadência metódica, delegando parcelas intensas nos agentes eletrónicos adjacentes, incitando ativa e autonomamente ciclos operacionais através das chamadas contíguas à ativação física das interações dos blocos IPs independentes — deflagrando processamento num crescendo orquestrado entre o preâmbulo do filtro fir(), avançando resolutamente na álgebra esmagadora do tensor com matmul() e finalizando os ordenamentos da memória no labirinto mutável subentendido na qsort()[1]. Numa demonstração imperiosa e irrefutável de verificação redundante arquitetural imposta sobre todo o ecossistema prototipado e final, cada salto evolutivo da rotina central acarreta iminentemente a intervenção inadiável da validação através de blocos e chamadas cruzadas consubstanciadas sob a designação fir_check(), matmul_check() e qsort_check(), cuja engenharia interna avalia o determinismo face aos padrões perfeitos matematicamente criados (referenciados na literatura pela designação formal de *golden models*)[1].

O Arquétipo Operacional Baseado no Polling de Checkbits

No desígnio exaustivo do auxílio diagnóstico durante a fase precoce de análise comportamental (*testbench RTL* pre-síntese), a totalidade da rede comunicacional baseada no eixo C-para-Verilog corporiza-se empiricamente além das paredes dos domínios da linguagem de software pura. Estas notificações propagam-se ativamente ao longo dos eixos e condutos físicos externos dos pinos e malhas do SoC, estrategicamente capturados no painel simulado a partir de um limite visual com máscara padronizada delimitada de forma inegociável nos

domínios do bloco de *Interface e Operacionalidade Geral* via porta lógica dedicada mprj_io[31:16].
Os traços ondulatórios lógicos expostos transmutam-se nestes domínios assumindo um formato inalterável e místico de sinaléticas binárias estáticas que, no vernáculo especializado das equipas académicas subjacentes, apelidaram coloquialmente de painéis de "checkbits"[1]:

Máscara Binária Formatada à Base Hexadecimal (16'h)	**Correspondência Direta do Estágio IP Associado**	**Interpretação Exata do Evento de Transição Capturado pela Máquina Observadora de Testbench**
AB00	Filtro Passivo e Contínuo FIR	Ratificação explícita de que a simulação orquestrada reconheceu o espoletar mecânico central do processo aritmético baseado em convoluções da árvore multiplicadora.
AB01	Filtro Passivo e Contínuo FIR	Acusação rigorosa da finalização global que consubstancia e confirma de antemão que o *pool* multiplicativo entrou no estado quiescente e o ciclo DMA regressou soberanamente para *idle*.
AB10 / AB11	Bloco Sistólico Matmul	Emissões intermédias associadas intrinsecamente ao gatilho inaugural / constatação do término laborioso da engrenagem subjacente da máquina sistólica tensorial assíncrona[1].
AB20 / AB21	Classificador Não-Linear Quick Sort	Disparo e extinção cabal atrelada aos ciclos imprevistos e irregulares

		alocados aos complexos exercícios na ramificação profunda dos bancos LIFO que modelizam artificialmente os percursos interativos impostos na RAM[1].
AB30 ... AB51	Sub-rotinas Analíticas de *Check*	Transmissões lógicas consubstanciais dedicadas exclusamente para confirmação protocolar final nas três valências do sistema (com a bifurcação entre balizamentos de arranque Start e colapso End da auditoria nativa cruzada em C)[1].

O engenhoso mecanismo analítico embutido sob forma de traços digitais e telégrafos não pode ser reduzido e classificado unicamente como excentricidade processual, resultando incisivamente numa contração abissal e transcendental do labor penoso inerente ao escrutínio visual de milhões de tramas microscópicas de comutação baseadas no relógio de sistema contidas dentro dos extensos registos topológicos das ondas digitais visualizáveis (habitualmente indexadas nos ficheiros main.vcd escrutinados exaustivamente por aplicações padronizadas na indústria como o projeto de software de visualização em massa GTKWave)[1]. A alocação contínua confere as âncoras espaciais e a indexação temporal absoluta no desenrolar e fluidez inquestionável da simulação da base teórica funcional concebida originalmente no fluxo dos diagramas matemáticos[1]. No seio das saídas dos terminais e das mensagens textuais alocadas e canalizadas conjuntamente com as evocações compiladas pelas diretivas de sistema do utilitário padronizado make sob os comandos interativos executados de raiz na plataforma primária do pacote de vivado de síntese algorítmica, o materializar indiscutível e tangível das *macros* textuais de rastreabilidade final (sob os dísticos formatados estritamente na forma global "Test start - FIR", progredindo iminentemente para a baliza temporal de "Test end - FIR" bem como sucedâneos similares) decreta, por meio de validade empírica suprema incontornável, o bloqueio irrevogável da auditoria com chancela dourada[1].

Considerações Analíticas Finais

Ao abrigo das premissas exigidas, o mapeamento final delineado nas análises do tecido funcional pertencente ao consórcio e amálgama intelectual gerado para este protótipo na alçada laboratorial baseada nas estruturas SoC do Caravel e das pontes do protocolo flexível AXI-Stream manifesta que os horizontes do paradigma da comutação passiva podem sobrelevar incrivelmente e atenuar a paralisia provocada pelo afamado estrangulamento

latente da interface entre o von Neumann puro e as limitações na largura de base operando nos limiares extremos físicos impostos à comunicação com a estática e rígida memória dos controladores primários de Bloco[1].
A robustez da matriz e do *bandwidth* alocado para suportar operatividade dos algoritmos submetidos reflete consistência perante as mais severas demandas matemáticas e operacionais em picos temporais exatos. Pese, todavia, o descompasso evidente e amplamente mitigável face aos paradigmas imutáveis, como na circunstância em que as avaliações no processo orgânico associado à expansão da profundidade multiplicativa imposta pela quantização imperiosa nas vias diretas transpostas do acelerador passivo FIR ditam invariavelmente uma gestão microscópica baseada estritamente no espetro do redimensionamento fixo contra anomalias originadas no crescimento logarítmico dos resguardos limitativos para a preservação contígua nos ciclos integrados finais dos algoritmos digitais acumuladores[4]. Em termos estritamente matemáticos sob o prisma do cômputo matricial bidimensional espacial em grelhas de matrizes paralelas matmul, exige-se sem hesitações a salvaguarda metódica nas formulações teóricas regidas rigorosamente sob o primado do modelo sistólico, impedindo ativamente que o teto computacional transposto atinja por incapacidade do motor DMA a desastrosa vala limitadora proveniente de um decréscimo drástico associado às características imprevisíveis enlaçadas na esparsidade do modelo de dados não densos subjacentes importados num volume desregulado dos bancos da memória associada brc0 que resultaria inevitavelmente na quebra infeliz imposta à simetria lógica[14].
A conversão arquitetural no mapeamento hardware das máquinas inferenciais finitas regidas pela incerteza associada estritamente às perturbações logarítmicas de $O(n^2)$ inerentes por definição formal à natureza não estruturada das metodologias heurísticas no *Quick Sort*, adverte impreterivelmente que a implementação rigorosa na adoção inorgânica acarreta e transfigura os princípios de tempo linear num conflito predatório, introduzindo perigosamente variações bruscas que incitam flutuações e entropias nefastas sobre o controlo do trânsito na base primária face aos pedidos submissos da CPU, criando vias operacionais lateralmente detetáveis vulneráveis a exploração baseadas no decurso exato de consumos associados e exigindo mitigações na base formal aleatorizada das estruturas da escolha elementar de forma protetiva e ativa[20]. Assim concluída, constata-se a exequibilidade prática e funcional assinalável de todas as componentes descritas que perfazem em conjunto um exemplo de modelação proficiente no ecossistema atual.

Works cited

1. kevin861222/112_SOC_final_project: soc lab final project - GitHub, https://github.com/kevin861222/112_SOC_final_project
2. Generative AI and Its Transformative Potential - MDPI, https://mdpi-res.com/bookfiles/book/10996/Generative_AI_and_Its_Transformative_Potential.pdf?v=1749036637
3. The Complete Guide to Hardware Pipeline Design with Backpressure, https://medium.com/@aiclab.official/the-complete-guide-to-hardware-pipeline-d

esign-with-backpressure-41d40ef6818d
4. Practical Considerations in Fixed-Point FIR Filter Implementations, http://www.digitalsignallabs.com/fir.pdf
5. 16+ Digital Signal Processing MATLAB Projects & Code (2026), https://www.matlabsolutions.com/project-ideas/digital-signal-processing-matlab-projects.php
6. Infinite Impulse Response Filter Structures in Xilinx FPGAs, https://docs.amd.com/api/khub/documents/iNmDMNbYApTu1sQ1gFn7tg/content
7. Energy-Efficient Fixed-Coefficient FIR Filters for Millimeter-Wave, https://lup.lub.lu.se/student-papers/record/9136449/file/9136450.pdf
8. FPGA FIR Development - Design by Ebrahim, https://designbyebrahim.com/projects/qmi/fpga_fir
9. FPGA based Design and Implementation of Higher Order FIR Filter, https://www.researchgate.net/publication/343448980_FPGA_based_Design_and_Implementation_of_Higher_Order_FIR_Filter_using_Improved_DA_Algorithm
10. UC San Diego - eScholarship.org, https://escholarship.org/content/qt7r89f4gx/qt7r89f4gx.pdf
11. The ZipCPU by Gisselquist Technology, https://zipcpu.com/blog.html
12. Digital-Signal-Processing-with-FPGA-Electrovolt.ir_.pdf, https://electrovolt.ir/wp-content/uploads/2017/07/Digital-Signal-Processing-with-FPGA-Electrovolt.ir_.pdf
13. Xilinx DS795 LogiCORE IP FIR Compiler v6.2, Data Sheet, https://docs.amd.com/api/khub/documents/bLbPTIafc5j_a_fN6xpo5g/content
14. Hardware Acceleration - Machine Learning Systems, https://mlsysbook.ai/vol1/hw_acceleration/hw_acceleration.html
15. (PDF) Best Practice Guide: Modern Accelerators - ResearchGate, https://www.researchgate.net/publication/353446204_Best_Practice_Guide_Modern_Accelerators
16. A FRAMEWORK FOR FAST ARCHITECTURE EXPLORATION OF, https://repositorio.pucrs.br/dspace/bitstream/10923/24474/1/000503662-Texto%2Bcompleto-0.pdf
17. Design Methodologies for Deep Learning Accelerators on ... - arXiv, https://arxiv.org/html/2311.17815v2
18. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based, https://www.csl.cornell.edu/~albonesi/research/papers/micro20-2.pdf
19. Paper-Collection-for-Machine-Learning-Hardware.md - GitHub, https://github.com/IAMAI/ML-Hardware-Collections/blob/main/Paper-Collection-for-Machine-Learning-Hardware.md
20. (PDF) Sort Data Faster: Comparing Algorithms - ResearchGate, https://www.researchgate.net/publication/394581428_Sort_Data_Faster_Comparing_Algorithms
21. Implementation and Performance Analysis of Qwimb Sort on - Scribd, https://www.scribd.com/document/1059268767/Implementation-and-Performance-Analysis-of-Qwimb-Sort-on-Derived-Datatypes
22. A Comparative Study of Sorting Algorithms with FPGA Acceleration,

https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S1405-554620190000100213
23. A QUICK SORT–BASED FRAMEWORK ... Sade et al., FJS FUDMA, https://fjs.fudutsinma.edu.ng/index.php/fjs/article/download/4289/3069/13314
24. Exploiting heterogeneous parallelism in software and hardware, https://www.repository.cam.ac.uk/bitstreams/cc1fbb91-b7b3-48d5-9df1-6873af351520/download
25. Accelerating Database Systems Using FPGAs: A Survey, https://readingxtra.github.io/docs/db_FPGA/08533481.pdf
26. TKAndyLiang/SOC_Lab: NYCU-2023-SoCLab - GitHub, https://github.com/TKAndyLiang/SOC_Lab
27. bol-edu/caravel-soc_fpga-lab - GitHub, https://github.com/bol-edu/caravel-soc_fpga-lab